WEEK 4: Uninformed Search Strategies (BFS, DFS, UCS)

Course: Artificial Intelligence
Lab Duration: 3 Hours

1. Introduction

Uninformed Search, also called Blind Search, refers to search techniques that explore a problem space without using additional domain-specific knowledge or heuristics.

These algorithms rely primarily on:

- **Initial State**: where the problem starts
- **Possible Actions**: what transitions are possible from the current state
- **Goal Test**: determines whether the desired state has been reached
- **Path Cost**: total cost accumulated along a path, particularly important for Uniform Cost Search

In real-world Artificial Intelligence systems, a problem can often be represented as a **graph of states and transitions**.

Examples include:

- Finding a route through a university campus
- Searching a public transportation network
- Exploring a computer's file system
- Finding a low-cost emergency route
- Exploring connected network devices
- Searching possible robot locations
- Exploring states in a puzzle or game

*The major difference between BFS, DFS, and UCS is **how they decide which state to explore next**.(**how they select the next node from the frontier**.)*

Algorithm	Main Idea	Typical Real-World Scenario
BFS	Explore level by level	Find a destination using the fewest transitions
DFS	Explore deeply before backtracking	Explore a file/folder hierarchy

UCS	Explore the lowest-cost path first	Find the minimum-cost emergency route

They systematically explore the state space using different strategies.

Key Components in All Uninformed Searches:

1. **State representation**: how each configuration of the problem is stored
2. **Frontier**: list of nodes to be expanded
3. **Explored set**: tracks visited nodes to avoid repetition
4. **Goal test**: checks if current node satisfies problem goal
5. **Node expansion**: generates successor states from the current node

Formulating a Real-World Problem as a Search Problem

Before implementing a search algorithm, an AI problem must be represented using states and transitions.

Consider an AI system searching for a room inside a university campus.

The problem can be represented as:

```
Initial State: FAST Main Gate

Actions: Move from one connected campus location to another

Goal: Target Room

State: A particular location inside the campus

Path: Sequence of locations visited to reach the target

Path Cost: Number of transitions, distance, time, or another defined cost
```

For example:

2. Representing Problems

Example 1: Unweighted Graph (for BFS/DFS)

Use Case: Finding a Target Room Inside a University Campus

Suppose an AI navigation assistant is helping a student find a **Target Room** after entering through the **FAST Main Gate**.

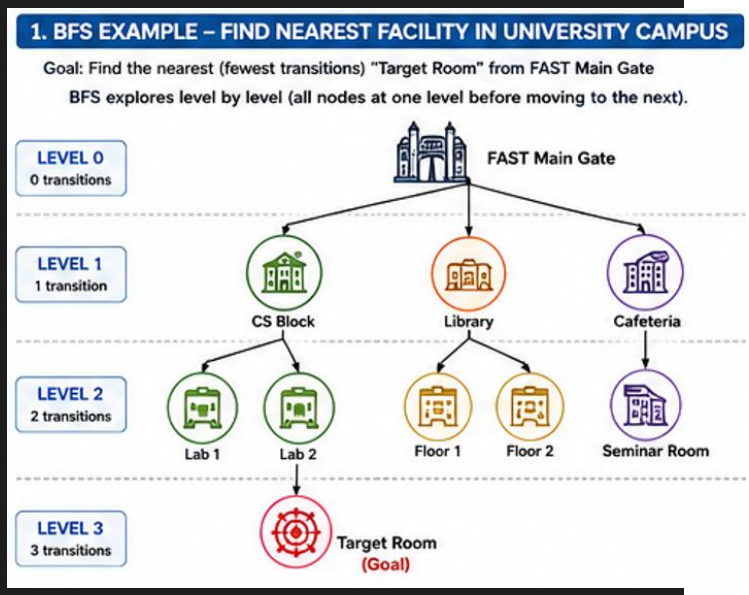
For this exercise, each connection between two locations is considered one equal-cost transition.

The campus can be represented as:

Python representation:

```
graph = {
    "FAST Main Gate": [
        "CS Block",
        "Library",
        "Cafeteria"
    ],
    "CS Block": [
        "Lab 1",
        "Lab 2"
    ],
    "Library": [
        "Floor 1",
        "Floor 2"
    ],
    "Cafeteria": [
        "Seminar Room"
    ],
    "Lab 1": [],
    "Lab 2": [
        "Target Room"
    ],

    "Floor 1": [],
    "Floor 2": [],
    "Seminar Room": [],
    "Target Room": []
}
```



The graph contains multiple branches.

This allows students to observe the difference between:

BFS → Level-by-level exploration

DFS → Deep exploration followed by backtracking

Example 2: Weighted Graph (for UCS)

Use Case: Emergency Route Planning

Suppose an AI-based emergency routing system needs to find a route from:

Shifa International Hospital (Source: Initial State)

to:

PIMS (Destination: Goal State)

Different road segments have different estimated travel-time costs.

For this lab exercise, the following illustrative travel-time costs are assigned:

Python representation:

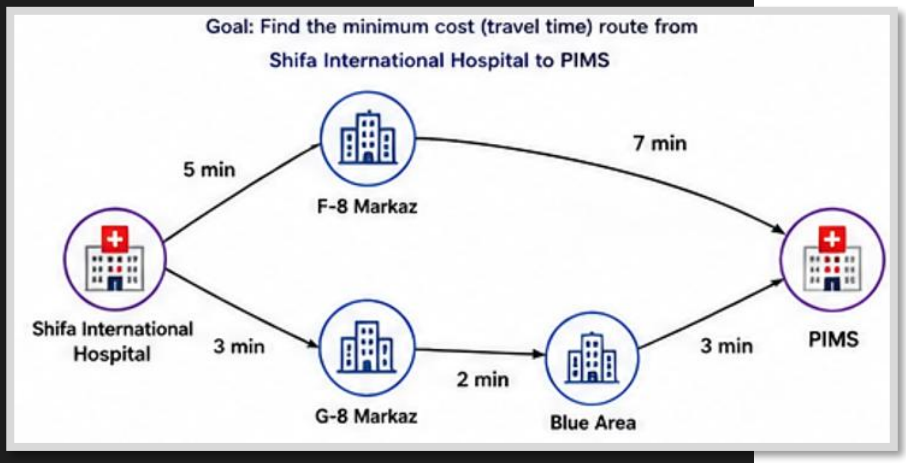
```
graph = {
    "Shifa International Hospital": [
        ("F-8 Markaz", 5),
        ("G-8 Markaz", 3)
    ],

    "F-8 Markaz": [
        ("PIMS", 7)
    ],

    "G-8 Markaz": [
        ("Blue Area", 2)
    ],

    "Blue Area": [
        ("PIMS", 3)
    ],

    "PIMS": []
}
```



Note: *The numerical travel times above are illustrative values used only to demonstrate UCS. They should not be interpreted as live traffic or actual travel-time estimates.*

The two possible routes are:

Route 1:

Shifa International Hospital → F-8 Markaz → PIMS

Total Cost: 5 + 7 = 12 minutes

and:

Route 2:

Shifa International Hospital → G-8 Markaz → Blue Area → PIMS

Total Cost: 3 + 2 + 3 = 8 minutes

Therefore, UCS should select Route 2 because its **total path cost is lower**.

Note: Algorithm logic is mostly the same; *only frontier and node expansion differs*.

3. Breadth First Search (BFS)

- Explores nodes **level by level**
- Finds the **shortest path in terms of edges** in unweighted graphs
- Uses **FIFO queue**

Real-World Use Case

BFS is useful when an AI system needs to find a solution requiring the **minimum number of transitions**, assuming all transitions have equal cost.

Examples include:

- Finding a room using the fewest connected locations inside a campus
- Finding the minimum number of transfers in an unweighted transportation graph
- Finding the minimum number of connections between two nodes in a network

BFS first explores:

- `Level 0`

  then:

- `Level 1`

  then:

- `Level 2`

  and so on.

  For the university navigation problem:

- `Level 0: FAST Main Gate`

  ↓

- `Level 1: CS Block | Library | Cafeteria`

  ↓

- `Level 2: Lab 1 | Lab 2 | Floor 1 | Floor 2 | Seminar Room`

  ↓

- `Level 3: Target Room`

- BFS does not completely follow one branch before considering the others.
- It finishes exploring the current level before moving deeper.

Data Structure:

```
from collections import deque

frontier = deque()
```

BFS Generic Code Pattern:

```
frontier.append(start_state)
visited = {start}#means we mark the start node as visited immediately, so it cannot be added to the queue again.
visited = set()# starts empty, so we would need to add the start node to visited separately before exploring neighbors.

while frontier:
    current = frontier.popleft()   # FIFO behavior
    if current == goal:
        break


    if current not in visited:
        visited.add(current)


        for neighbor in graph[current]:
            if neighbor not in visited:
                frontier.append(neighbor)
```

Applied Example

To search for the Target Room:

start_state = "FAST Main Gate"

goal = "Target Room"

The BFS frontier initially contains: [FAST Main Gate]

After expanding the starting state: [CS Block, Library, Cafeteria]

These are all **Level 1** states.

After expanding the Level 1 states, the frontier contains states from **Level 2**:

[Lab 1, Lab 2, Floor 1, Floor 2, Seminar Room]

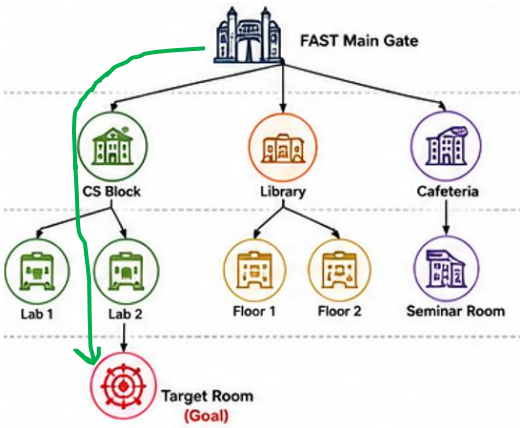
When Lab 2 is expanded: Target Room is discovered at Level 3.

Therefore:

BFS Path:

FAST Main Gate → CS Block → Lab 2 → Target Room

Number of transitions: 3

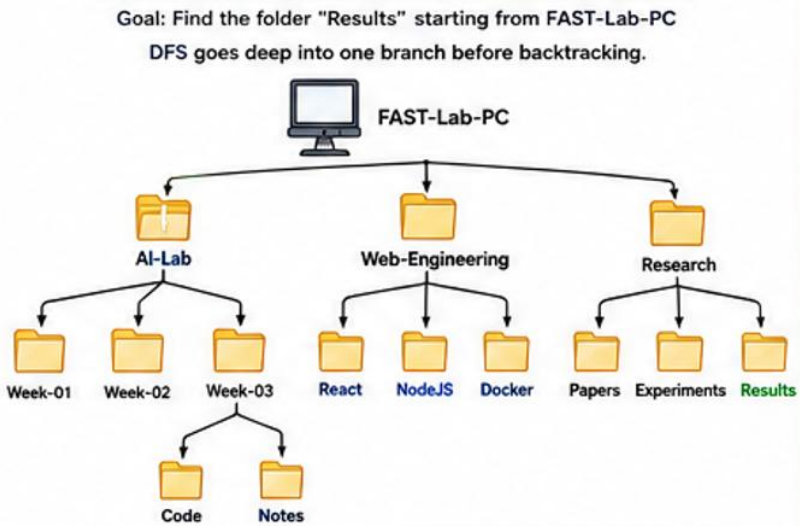


4. Depth First Search (DFS)

- Explores as **deep as possible** along a path before backtracking
- Memory efficient
- **Does not guarantee shortest path**

Real-World Use Case

- DFS is useful when an AI system needs to deeply explore a hierarchical or branching structure.
- A relatable example is searching a university laboratory computer's file system for a particular folder or file.
- Suppose the computer contains:



Data Structure:

```
stack = [start]
```

DFS Generic Code Pattern:

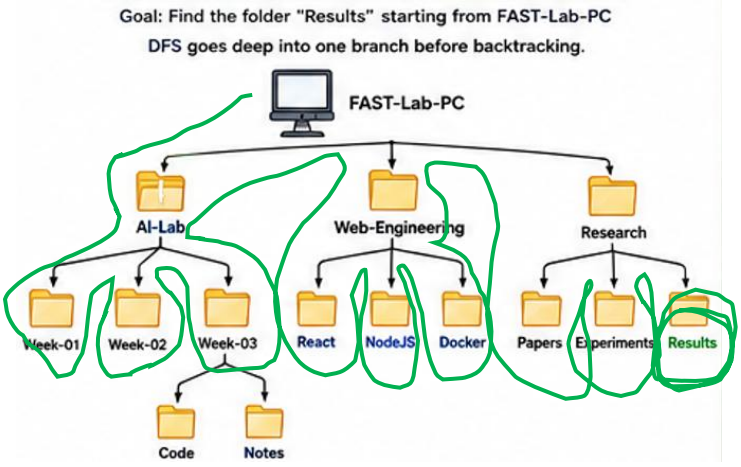
```
stack.append(start_state)
visited = set()

while stack:
    current = stack.pop()   # LIFO behavior

    if current == goal:
        break

    if current not in visited:
        visited.add(current)

        for neighbor in graph[current]:
            if neighbor not in visited:
                stack.append(neighbor)
```



Applied Example

Start: FAST-Lab-PC

Goal: Results

One possible DFS exploration is:

FAST-Lab-PC → AI-Lab → Week-01 → backtrack → Week-02 → backtrack → Week-03 → backtrack → Web-Engineering → React → backtrack → NodeJS → backtrack → Docker → backtrack → Research → Papers → backtrack → Experiments → backtrack → **Results**

The exact traversal order depends on the order in which neighboring nodes are stored.

The important concept is: Go deep

↓

Reach dead end

↓

Backtrack

↓

Try another branch

5. Uniform Cost Search (UCS)

- Expands the node with **lowest total path cost** first
- Guarantees **optimal solution**
- Generalization of BFS for weighted graphs

Real-World Use Case

UCS is useful when different actions have different costs.

Examples include:

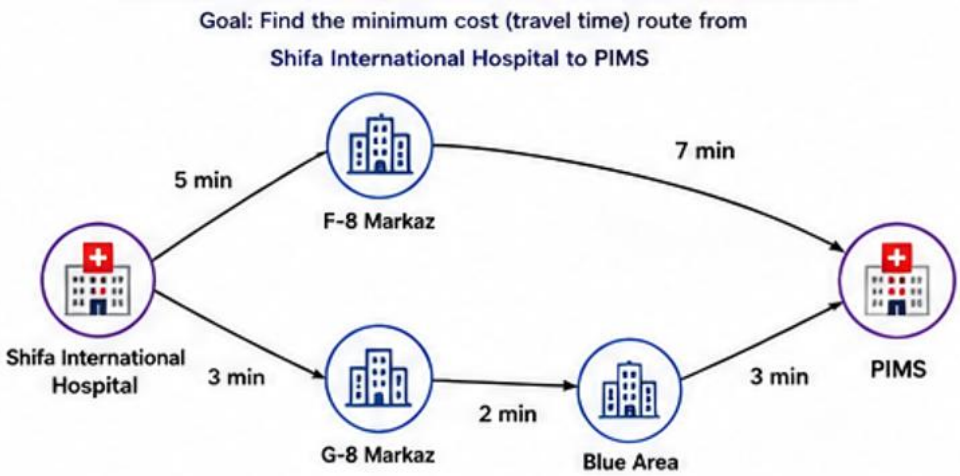
- Emergency route planning
- Delivery route optimization
- Transportation cost optimization
- Network routing where links have different costs

For this lab, consider:

Shifa International Hospital as the starting location and:

PIMS as the destination.

The road costs are:



UCS does not simply count the number of edges.

Instead, it calculates:

Total path cost = sum of all edge costs

Therefore:

Shifa → F-8 Markaz → PIMS

5 + 7 = 12

while:

Shifa → G-8 Markaz → Blue Area → PIMS

3 + 2 + 3 = 8

UCS selects:

Shifa International Hospital → G-8 Markaz → Blue Area → PIMS

because: **8 < 12**

Data Structure:

```
import heapq
priority_queue = []
```

UCS Node Representation:

```
(cost_so_far, current_state)
```

UCS Generic Code Pattern:

```
heapq.heappush(priority_queue, (0, start))
visited = {}


while priority_queue:
    cost, current = heapq.heappop(priority_queue)


    if current == goal:
        break
```

```
    if current not in visited or cost < visited[current]:
        visited[current] = cost

        for neighbor, edge_cost in graph[current]:
            new_cost = cost + edge_cost
            heapq.heappush(priority_queue, (new_cost, neighbor))
```

6. Path Tracking (Common for BFS/DFS/UCS)

To reconstruct a solution path, maintain a **parent dictionary**:

```
parent[child] = current
```

For example, for the emergency route:

```
parent = {
    "G-8 Markaz":
        "Shifa International Hospital",
    "Blue Area":
        "G-8 Markaz",
    "PIMS":
        "Blue Area"
}
```

Reconstruct the path:

```
path = []
node = goal
while node is not None:
    path.append(node)
    node = parent.get(node)
path.reverse()
```

Tip: Always initialize ***parent[start] = None***.

This allows the path reconstruction loop to know when it has reached the starting state.

7. Comparison Table

Feature	BFS	DFS	UCS
Data structure	Queue	Stack	Priority Queue
Explores	Level by level	Deep first	Least cost first
Completeness	Yes*	No	Yes*
Optimal	Yes (unweighted)	No	Yes*
Memory	High	Low	Moderate
Main Use case	Shortest path (in terms Of edges: Transitions)	Deep exploration	Weighted shortest path
Real-world example	University facility search	File-system exploration	Emergency route planning
Main decision factor	Depth / level	Depth	Accumulated cost

*Under the standard assumptions for each algorithm.

8. Common Mistakes to Avoid

- Marking nodes as visited **too early or too late**
- Using incorrect data structure for **frontier**
- Ignoring **cost** update in UCS
- Assuming BFS works on weighted graphs
- Expecting DFS to give shortest path
- Forgetting that BFS explores **level by level**
- Confusing the number of transitions with total path cost
- Forgetting to track parent relationships when the actual route/path is required
- Failing to handle cycles in real-world graphs

Important Reminder

- **BFS:** Minimum number of edges
- **UCS:** Minimum accumulated cost
- **DFS:** Deep exploration + backtracking

These are different objectives.

9. Lab Skeleton Code

```
def search_algorithm(graph, start, goal):
    # Initialize frontier
    # Initialize visited/explored
    # Loop until frontier is empty
    # Pop node according to algorithm
    # Goal test
    # Expand neighbors
    # Track parent for path reconstruction
    pass
```

You need to implement BFS/DFS/UCS by adjusting frontier and expansion logic.

For your implementation:

BFS → Queue

DFS → Stack

UCS → Priority Queue

*The graph representation may **remain similar**, but the mechanism for selecting the **next state changes**.*

Note the Key Points about the Frontier:

1. **Purpose:**
    - It stores **all the nodes that are reachable but haven't been expanded yet**.
    - Ensures that the search algorithm knows **where to go next**.
1. **Behavior depends on the algorithm:**

Algorithm	Frontier Data Structure	How nodes are selected
BFS	Queue (FIFO)	Oldest node first (level-order)
DFS	Stack (LIFO)	Most recent node first (deepest path)
UCS	Priority Queue (heapq)	Node with **lowest path cost** first

2. **Difference from Explored/Visited Set:**

- **Frontier** = nodes generated but not expanded
- **Explored/Visited** = nodes that have already been expanded

3. **Example Analogy:**

Imagine an AI navigation assistant exploring a university building.

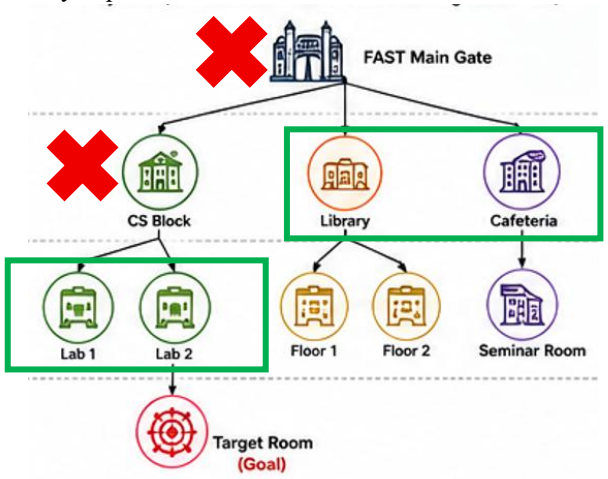
Frontier: Locations discovered but not explored yet

Visited: Locations that the AI has already explored

For example:

Visited:
FAST Main Gate
CS Block

Frontier:
Library
Cafeteria
Lab 1
Lab 2



Visualization of Graph:

```
import networkx as nx
import matplotlib.pyplot as plt

# Create an undirected graph
G = nx.Graph()

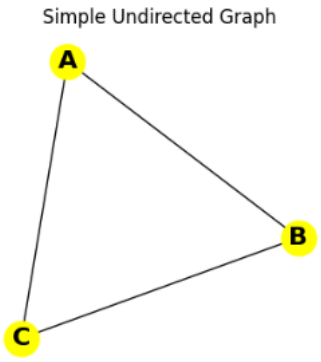

G.add_edges_from([
    ('A', 'B'),
    ('B', 'C'),
    ('A', 'C'),
])

# Draw the graph
plt.figure(figsize=(3, 3))
pos = nx.spring_layout(G, seed=42)  # seed for consistent layout

nx.draw(G, pos, with_labels=True, node_color='yellow',
        node_size=500, font_size=16, font_weight='bold')
plt.title("Simple Undirected Graph")
plt.show()
```

if you already define your graph like:
graph={
"FAST": ["NUST", "Library"],
...
}
then,

```
for node in graph:
    for neighbor in graph[node]:
        G.add_edge(node, neighbor)
```



Highlight solution path

```
path_edges = list(zip(path, path[1:]))

nx.draw_networkx_edges(G,pos,edgelist=path_edges,width=4,
    arrows=True,
    arrowsize=25,
    edge_color='r'
)
```

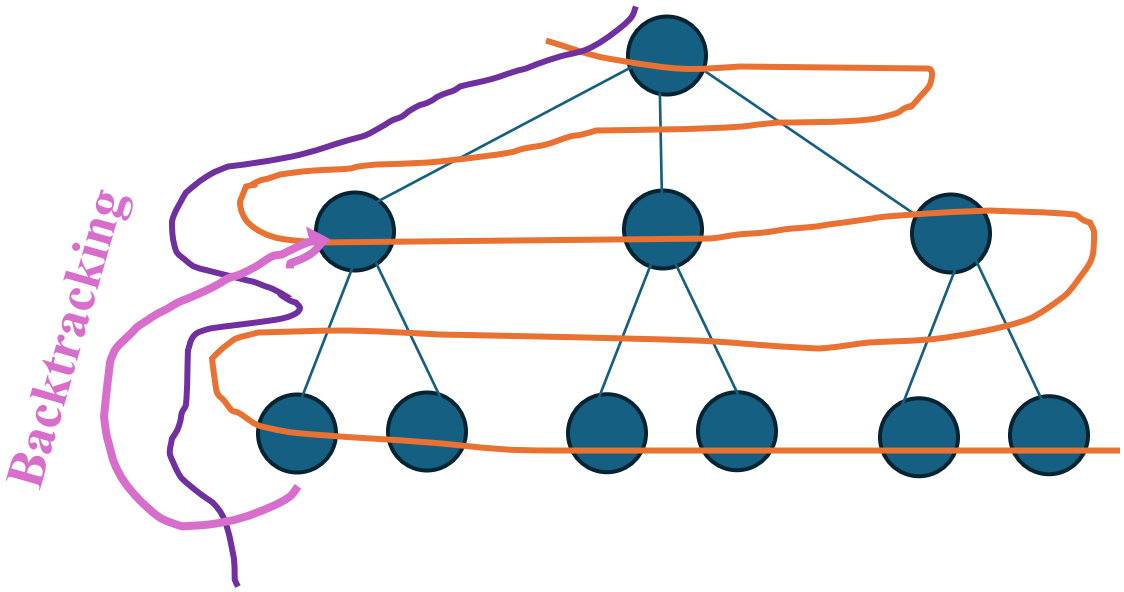
For Explanation visit: https://www.kaggle.com/code/alireza151/networkx-tutorial

EXECUTION FLOW (DRY RUN)

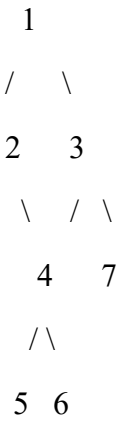
GRAPH TRAVERSAL:

The process of visiting and exploring a graph for processing is call as graph traversal.

1. Breadth First Search **(BFS) level ordering**
2. Depth First Search **(DFS)**



BFS:



1	2	3	4	7	5	6

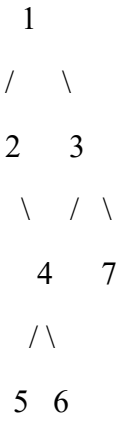
Dry Run:

1 take it out from queue and explore it -> 2,3 connect so insert these to the queue

Similarly now take out 2 and explore it -> 4 so add 4 in queue
at last,

1 2 3 4 7 5 6

DFS: (Stack Data structure use to back track)



3
4
2
1

Dry Run:

1 visited explore it -> either 2 or 3 let say we take 2 then

1 will be added to the stack, then 2 -> 4

So 2 will be added to stack, 4 -> 5 or 6 let say we take 5

And now from 5 no other way/node to travel so we back track using stack we get to know where we have to travel i.e. 4 last element of 4

Then we see we have 6 so added it to sequence and 4 has been visited so we added it to the stack: 1 2 4 5 6 , again 6 has no way further so back track again 4 connected to 3 so add it to seq 1 2 4 5 6 3 : 4 added to stack then from 3 to 7 : 1 2 4 5 6 7 , 3 added to stack, 7 has no further connected node so back track this time we have 3 and 3 is connected to 1 which was already been visited so add 3 back to stack back tracking from values inside the stack if no other options left we got our traversal sequence:

at last,

1 2 4 5 6 3 7

Depth First Search:

Stack/recursive approach

Steps:

- Start at 1
- Visit 2 → visit 4 → visit 5 → backtrack → visit 6 → backtrack
- Back to 1, now visit 3 → visit 7

Result (one possible DFS path):

DFS: 1 2 4 5 6 3 7